

RÉPUBLIQUE TUNISIENNE
Ministère de l'Enseignement Supérieur et de la
Recherche Scientifique
Université de Carthage
Institut National des Sciences Appliquées et de
Technologie

Rapport de stage
Spécialité : Informatique Industrielle et Automatique

**Intégration et validation d'une interface web
de supervision et de commande du robot
AMR-X**

Envoi de destinations à Nav2 depuis le dashboard et suivi
de la position du robot simulé sous ROS 2 / Gazebo

Réalisé par :
Yassine Jelassi

Organisme d'accueil : [Organisme d'accueil]

Projet : AMR-X – *Autonomous Modular Robot – Extended*

Encadrant(e) à l'entreprise [Nom et prénom]	Encadrant(e) à l'INSAT [Nom et prénom]

Période du stage : [du ... au ...]

Année universitaire : [20.. / 20..]

Remerciements

Je tiens à exprimer ma gratitude à [Nom de l'encadrant(e)], mon encadrant(e) au sein de [organisme d'accueil], pour son accompagnement tout au long de ce stage, sa disponibilité et la pertinence de ses retours.

Je remercie également [Nom de l'encadrant(e) académique], pour son suivi pédagogique et ses orientations.

Mes remerciements s'adressent aussi à l'ensemble de l'équipe du projet AMR-X, dont les travaux sur la simulation, la navigation et le dashboard ont constitué la base de ce stage, ainsi qu'au corps professoral et administratif de l'INSAT pour la formation reçue.

Table des matières

Liste des acronymes	vi
Glossaire	vii
Introduction générale	1
I Contexte général du projet	2
1 Présentation de l'organisme d'accueil	2
2 Le projet AMR-X	2
3 Existant au début du stage	3
3.1 Simulation et navigation	3
3.2 Dashboard web	3
3.3 Limites de l'existant	3
4 Problématique	4
5 Cahier des charges	4
5.1 Besoins fonctionnels	4
5.2 Besoins non fonctionnels	5
5.3 Périmètre et limites	5
6 Environnement de travail	5
7 Démarche adoptée	6
II Conception de la solution	7
1 Architecture générale	7
2 Choix d'une passerelle dédiée	8
3 Protocole de communication	8
3.1 Service de soumission	8
3.2 État de navigation	9
3.3 Séquence d'une navigation	9
4 Machine d'états d'une demande	10
5 Gestion des repères	10
6 Vérification des destinations	11
7 Règles de sûreté et de fraîcheur	11
III Réalisation	13

1	Organisation des fichiers	13
2	Passerelle ROS 2	14
2.1	Paramètres	14
2.2	Fonctionnement	14
2.3	Fichier de lancement	15
3	Interface web	15
3.1	Logique de navigation	15
3.2	Panneau de navigation et carte	15
4	Lancement de la chaîne complète	16
5	Tests automatisés	17
6	Problèmes rencontrés et corrections	17
IV Validation en simulation		19
1	Environnement et méthode d'essai	19
2	Vérifications préalables	20
3	Mise en évidence d'un décalage de repères	21
3.1	Constat	21
3.2	Analyse	21
3.3	Correction	21
4	Résultats après correction	22
4.1	Cohérence des positions	22
4.2	Résultats par scénario	23
4.3	Illustration d'une navigation réussie	23
4.4	Analyse	23
5	Points restant à valider	25
6	Limites	25
Conclusion générale et perspectives		27

Table des figures

I.1	Briques existantes au début du stage.	4
II.1	Architecture de la chaîne de commande et de retour (en rouge, le nœud développé).	7
II.2	Séquence d'une navigation réussie.	9
II.3	Phases d'une demande de navigation (en jaune, les phases actives).	10
III.1	Page <i>Live map</i> : robot <code>amr_x</code> affiché à sa position ROS sur le plan du monde <code>warehouse_harmonic</code>	16
III.2	Séquence de démarrage du script <code>run-all.sh</code>	17
IV.1	Robot AMR-X simulé dans l'entrepôt Gazebo.	20
IV.2	Origine du repère <code>map</code> dans le monde Gazebo.	22
IV.3	Navigation réussie : déplacement du robot vu dans le dashboard et dans Gazebo.	24
IV.4	Résultat de la navigation affiché dans le panneau de flotte du dashboard.	24

Liste des tableaux

I.1	Besoins fonctionnels.	4
I.2	Environnement logiciel du stage.	5
II.1	Contenu de l'état de navigation publié par la passerelle.	9
II.2	Règles de sûreté et de fraîcheur des données.	11
III.1	Principaux fichiers ajoutés.	13
III.2	Paramètres de la passerelle.	14
III.3	Problèmes rencontrés lors de l'intégration et corrections.	18
IV.1	Vérifications préalables sous Ubuntu.	20
IV.2	Écart entre la position affichée et la position Gazebo, sans correction de repère.	21
IV.3	Écart entre la position affichée et la position Gazebo, après correction.	22
IV.4	Résultats mesurés par scénario, après correction des repères.	23

Liste des acronymes

AMCL	Adaptive Monte Carlo Localization
AMR	Autonomous Mobile Robot
API	Application Programming Interface
JSON	JavaScript Object Notation
MPPI	Model Predictive Path Integral
Nav2	Navigation 2 (pile de navigation ROS 2)
QoS	Quality of Service
REST	Representational State Transfer
ROS 2	Robot Operating System 2
RViz	ROS Visualization
SDF	Simulation Description Format
SLAM	Simultaneous Localization and Mapping
SPA	Single Page Application
SVG	Scalable Vector Graphics
TF	Transform (arbre des repères ROS)
UI	User Interface

Glossaire

Acquittement	Réponse d'un service ROS indiquant que la demande a été reçue et acceptée ou refusée. Un acquittement positif ne signifie pas que la destination est atteinte.
Costmap	Grille d'occupation utilisée par Nav2, dont chaque cellule contient un coût de 0 (libre) à 100 (occupé) ou -1 (inconnu).
Dashboard	Application web existante du projet AMR-X, composée d'un frontend React et d'un backend FastAPI, servant à la supervision de la flotte.
Donnée périmée	État de navigation reçu depuis plus de 2,5 s ; le dashboard le considère comme non fiable et bloque toute nouvelle commande.
Monde SDF	Description de l'environnement simulé chargée par Gazebo ; le dashboard en extrait un plan 2D pour l'affichage.
Passerelle	Nœud ROS 2 <code>dashboard_navigation</code> développé pendant le stage, intermédiaire unique entre le navigateur et Nav2.
Phase	État courant d'une demande de navigation : <code>idle</code> , <code>sending</code> , <code>navigating</code> , <code>canceling</code> , <code>succeeded</code> , <code>canceled</code> , <code>failed</code> ou <code>rejected</code> .
Repère map	Repère global de Nav2, défini par la carte chargée. Il ne coïncide pas nécessairement avec le repère du monde Gazebo.
rosbridge	Serveur exposant les topics et services ROS 2 au navigateur à travers une connexion WebSocket (port 9090).

Introduction générale

Un robot mobile autonome n'est exploitable que si un opérateur peut lui confier une tâche et suivre son exécution. Dans un entrepôt ou un hôpital, cet opérateur ne travaille pas dans un terminal ROS : il utilise une interface de supervision qui affiche la position des robots, lui permet de désigner une destination et l'informe du résultat. La qualité de cette interface dépend moins de son apparence que de la fidélité des informations affichées : une position fausse, un ordre envoyé deux fois ou un succès annoncé à tort peuvent conduire l'opérateur à une mauvaise décision.

Le présent stage s'inscrit dans le projet AMR-X, une plateforme de robot mobile modulaire développée sous ROS 2. Au début du stage, le projet disposait déjà d'une simulation Gazebo du robot dans un entrepôt, d'une pile de navigation Nav2 configurée et d'un dashboard web de supervision. Ces trois briques fonctionnaient séparément : le dashboard affichait une carte du monde mais ne pouvait ni connaître la position réelle du robot simulé, ni lui envoyer une destination. L'objectif du stage est de relier ces briques afin que l'opérateur puisse, depuis le dashboard, choisir une destination, commander le robot simulé dans Gazebo et voir sa position et le résultat de la navigation dans le navigateur.

La démarche adoptée consiste d'abord à analyser l'existant et à définir un protocole de communication sûr entre le navigateur et Nav2, puis à développer une passerelle ROS 2 et les composants correspondants du dashboard, et enfin à valider l'ensemble dans la simulation par des essais mesurés. Une attention particulière est portée à la cohérence entre le repère de la carte Nav2 et celui du monde Gazebo, qui s'est révélée déterminante lors des essais.

Ce rapport comporte quatre chapitres. Le premier présente le contexte du projet AMR-X, l'existant, la problématique et le cahier des charges. Le deuxième décrit la conception de la solution : architecture, protocole de communication, machine d'états et transformation des repères. Le troisième détaille la réalisation de la passerelle ROS 2, de l'interface web et de l'outillage de lancement, ainsi que les problèmes rencontrés. Le quatrième présente le protocole de validation et les résultats mesurés en simulation.

Chapitre I

Contexte général du projet

Introduction

Ce chapitre situe le stage dans son environnement. Il présente d’abord l’organisme d’accueil et le projet AMR-X, puis décrit les briques logicielles existantes au début du stage et leurs limites. À partir de cet existant, la problématique et le cahier des charges sont définis. Le chapitre se termine par l’environnement de travail et la démarche adoptée.

1 Présentation de l’organisme d’accueil

[Présenter ici l’organisme d’accueil : activité, date de création, implantation, équipe dans laquelle le stage a été réalisé.]

2 Le projet AMR-X

AMR-X (*Autonomous Modular Robot – Extended*) est un projet de plateforme mobile modulaire : une base robotisée commune reçoit des modules fonctionnels interchangeable selon l’application visée (logistique d’entrepôt, milieu hospitalier, etc.). Le dépôt du projet regroupe la définition produit, les études de marché, la mécanique, l’électronique, la partie robotique ROS 2 et les outils web.

La partie robotique cible l’environnement Ubuntu 24.04, ROS 2 Jazzy et Gazebo Harmonic. Elle comprend notamment la description du robot, sa simulation dans plusieurs mondes (entrepôt, hôpital), la cartographie par SLAM et la navigation autonome avec Nav2. Le projet est mené par plusieurs équipes de stagiaires qui travaillent sur des parties différentes du dépôt ; le travail présenté ici devait donc s’intégrer à l’existant sans le reconstruire.

3 Existant au début du stage

3.1 Simulation et navigation

Le robot AMR-X est simulé dans Gazebo Harmonic. Le monde retenu pour ce stage est l'entrepôt `warehouse_harmonic`, dans lequel le robot apparaît à une position de départ vérifiée, $(-1,2639; -3,9049)$ m. Le robot est équipé de deux LiDAR dont les mesures sont fusionnées sur un topic unique, d'une caméra et d'une centrale inertielle.

La navigation repose sur Nav2 : localisation AMCL sur une carte d'occupation enregistrée au préalable, planificateur global NavFn et contrôleur local MPPI. Une destination peut être envoyée à Nav2 depuis RViz ou par l'action ROS 2 `/navigate_to_pose`.

3.2 Dashboard web

Le dashboard est une application web de supervision composée de deux parties :

- un **frontend** React (Vite) organisé en pages : vue d'ensemble, robots, missions, carte en direct (*Live map*), alertes, modules, téléopération, paramètres et gestion des utilisateurs ;
- un **backend** FastAPI avec SQLAlchemy et Alembic, qui gère les comptes (avec approbation par un administrateur), les robots, les missions et les alertes dans une base de données relationnelle.

La page *Live map* affiche le monde SDF choisi sous forme de plan 2D (SVG) ou de vue isométrique 3D. Les mondes sont extraits automatiquement des fichiers SDF du dépôt dans un manifeste JSON (six mondes). Une connexion rosbridge existait déjà pour certaines pages.

3.3 Limites de l'existant

L'analyse de l'existant a fait apparaître les limites suivantes :

- la *Live map* n'affichait pas la position ROS du robot : les robots visibles provenaient de données de démonstration ou de la base de données ;
- en cas d'erreur de l'API, la page basculait implicitement sur les robots de démonstration, sans que l'utilisateur puisse distinguer des données réelles ;
- la vue 3D appliquait un décalage arbitraire de $(-12; -8)$ et animait un robot et un trajet décoratifs sans lien avec la simulation ;
- les boutons de la page *Missions* enregistrent des missions dans la base mais ne pilotent pas Nav2 ;
- aucun chemin ne permettait d'envoyer une destination au robot depuis le navigateur ni d'en connaître le résultat.

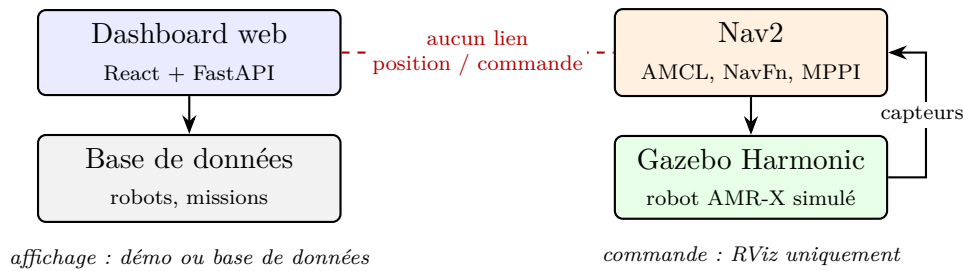


FIGURE I.1 – Briques existantes au début du stage.

4 Problématique

Le dashboard et la simulation formaient deux mondes séparés. La problématique du stage peut se formuler ainsi : *comment permettre à un opérateur de commander la navigation du robot AMR-X simulé depuis le dashboard web existant, tout en lui garantissant que la position et l'état affichés correspondent réellement à ceux du robot ?*

Plusieurs difficultés en découlent. Le navigateur communique en WebSocket et ne parle pas directement ROS 2. Une commande envoyée à travers le réseau peut être perdue, dupliquée ou arriver alors que le robot exécute déjà une autre destination. Le plan affiché dans le navigateur est construit à partir du monde SDF de Gazebo, alors que Nav2 raisonne dans le repère de sa carte. Enfin, l'interface doit rester honnête lorsque les données deviennent indisponibles, par exemple si la simulation est en pause ou si la connexion est perdue.

5 Cahier des charges

5.1 Besoins fonctionnels

Le tableau I.1 résume les besoins fonctionnels retenus.

Réf.	Besoin
BF1	Afficher en direct la position et l'orientation du robot simulé sur la <i>Live map</i> (vues 2D et 3D).
BF2	Choisir une destination par clic sur la carte 2D ou par saisie de X, Y (m) et du cap (°).
BF3	Envoyer la destination à Nav2 et obtenir un acquittement explicite de la demande.
BF4	Suivre l'état de la navigation : envoi, navigation, distance restante, résultat final.
BF5	Annuler la navigation en cours et afficher la confirmation d'annulation.
BF6	Refuser les destinations invalides (hors carte, occupées, non finies, mauvais monde) avec un message explicite.

TABLEAU I.1 – Besoins fonctionnels.

5.2 Besoins non fonctionnels

- **Fidélité** : ne jamais afficher de position ou de résultat inventé ; le succès provient uniquement du résultat Nav2.
- **Sûreté des commandes** : un seul but actif à la fois, pas de renvoi automatique d'une commande dont le résultat est incertain.
- **Détection des données périmées** : bloquer les commandes si l'état n'a pas été reçu depuis 2,5 s.
- **Intégration** : réutiliser le dashboard, Nav2 et la simulation existants sans les reconstruire ; conserver l'authentification existante.
- **Reproductibilité** : pouvoir lancer et vérifier l'ensemble de la chaîne par des commandes documentées.

5.3 Périmètre et limites

Le périmètre a été volontairement restreint à un robot, un serveur de navigation et un monde actif à la fois. Le travail porte sur la simulation : la passerelle refuse les destinations lorsque le temps simulé n'est pas utilisé, afin d'éviter toute commande d'un robot physique. La pause et la reprise d'une navigation, l'arbitrage avec d'autres sources de commande (RViz, téléopération) et l'authentification des commandes au niveau de rosbridge sont hors périmètre. Les navigations lancées depuis la *Live map* ne sont pas enregistrées dans la base de données.

6 Environnement de travail

Le tableau I.2 présente les outils utilisés.

Élément	Version / outil
Système d'exploitation	Ubuntu 24.04
Intergiciel robotique	ROS 2 Jazzy
Simulateur	Gazebo Harmonic
Navigation	Nav2 (AMCL, NavFn, MPPI)
Pont web	rosbridge_server 2.7.1 (WebSocket, port 9090)
Frontend	React 19, Vite 8, roslib 2.1
Backend	FastAPI, SQLAlchemy, Alembic
Environnement JavaScript	Node.js 20
Gestion de version	Git, GitHub

TABLEAU I.2 – Environnement logiciel du stage.

7 Démarche adoptée

Le travail a été conduit en quatre étapes :

1. analyse de l'existant (dashboard, simulation, configuration Nav2) et définition du parcours utilisateur ;
2. conception du protocole entre le navigateur et Nav2 et de la gestion des repères ;
3. développement de la passerelle ROS 2 et des composants du dashboard, avec des tests unitaires sans ROS ;
4. intégration sous Ubuntu, correction des problèmes observés et validation par des essais mesurés dans Gazebo.

Conclusion

Ce chapitre a présenté le projet AMR-X et l'existant au début du stage : une simulation, une pile Nav2 et un dashboard fonctionnels mais non reliés. Le cahier des charges fixe l'objectif de commander le robot depuis le dashboard avec des informations fidèles. Le chapitre suivant décrit la conception retenue pour y parvenir.

Chapitre II

Conception de la solution

Introduction

Ce chapitre présente la conception retenue pour relier le dashboard à Nav2. Il décrit l'architecture générale, justifie le choix d'une passerelle ROS 2 dédiée, puis détaille le protocole de communication, la machine d'états d'une demande de navigation, la gestion des repères et les règles de sûreté.

1 Architecture générale

La figure II.1 présente la chaîne retenue. Le navigateur communique avec ROS 2 par `rosbridge`, au moyen d'une connexion `WebSocket`. Il n'appelle pas `Nav2` directement : toutes les commandes passent par un nœud intermédiaire, `dashboard_navigation`, qui vérifie la demande, dialogue avec `Nav2` par l'action `/navigate_to_pose` et publie en retour un état synthétique. La position du robot est lue dans l'arbre TF (transformation `map` $\rightarrow$ `base_link`), qui combine la localisation `AMCL` et l'odométrie.

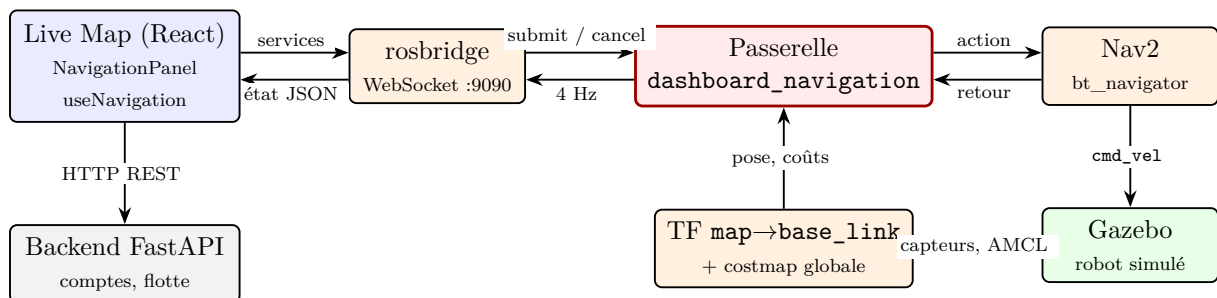


FIGURE II.1 – Architecture de la chaîne de commande et de retour (en rouge, le nœud développé).

2 Choix d'une passerelle dédiée

Deux approches étaient possibles pour envoyer une destination depuis le navigateur : publier directement un message sur le topic `/goal_pose` ou appeler l'action Nav2 depuis le navigateur, ou bien passer par un nœud intermédiaire. La première approche est simple mais ne fournit aucun accusé de réception : le navigateur ne sait pas si la commande a été reçue, refusée ou perdue. La seconde place dans le navigateur toute la logique de vérification, qui peut alors être contournée ou dupliquée par plusieurs onglets.

La passerelle dédiée a été retenue pour les raisons suivantes :

- chaque commande reçoit une réponse explicite (*acceptée* ou *refusée* avec un motif) ;
- les vérifications (monde, coordonnées, carte, disponibilité de Nav2) sont faites côté ROS, au plus près des données ;
- un seul but peut être actif à la fois, quel que soit le nombre de navigateurs ;
- l'état publié est unique et partagé : tous les clients voient la même information.

3 Protocole de communication

3.1 Service de soumission

La soumission utilise une interface de service dédiée, définie dans le paquet `navigation_msgs` :

```
# Coordinates in the dashboard's configured SDF world (metres, radians).
string request_id
string world_id
float64 x
float64 y
float64 yaw
---
bool accepted
string message
```

Le champ `request_id` est un identifiant aléatoire de 32 caractères hexadécimaux généré par le navigateur. Il permet à la passerelle de reconnaître une demande déjà reçue et de ne pas l'exécuter deux fois. Le champ `world_id` identifie le monde affiché dans le dashboard ; la passerelle refuse la demande s'il ne correspond pas au monde simulé. L'annulation utilise le service standard `std_srvs/srv/Trigger`.

Un acquittement positif signifie uniquement « demande reçue et transmise à Nav2 ». Le résultat réel de la navigation n'est connu qu'à travers l'état publié.

3.2 État de navigation

La passerelle publie à 4 Hz sur `/dashboard/navigation/state` un message `std_msgs/String` contenant un objet JSON versionné (tableau II.1). Ce choix évite de créer un type de message supplémentaire côté navigateur tout en gardant un format validé strictement par le frontend.

Champ	Contenu
version	Version du format (1) ; toute autre valeur est rejetée par le frontend.
world_id	Monde configuré sur la passerelle.
request_id	Identifiant de la dernière demande acceptée.
phase, detail	Phase de la demande et message lisible.
ready, unavailable_reason	Disponibilité pour une nouvelle destination et motif sinon.
pose	Position (x, y) et cap du robot dans le repère du monde, ou <code>null</code> .
target	Destination en cours, dans le repère du monde.
distance_remaining	Distance restante fournie par le retour de Nav2.

TABLEAU II.1 – Contenu de l'état de navigation publié par la passerelle.

3.3 Séquence d'une navigation

La figure II.2 montre les échanges lors d'une navigation réussie.

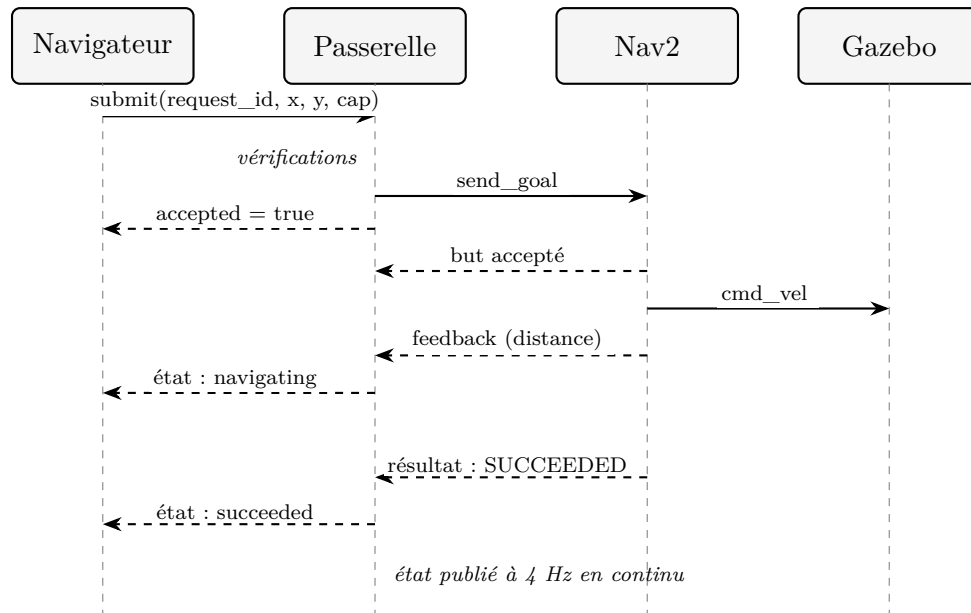


FIGURE II.2 – Séquence d'une navigation réussie.

4 Machine d'états d'une demande

Chaque demande suit la machine d'états de la figure II.3. Les phases **sending**, **navigating** et **canceled** sont dites *actives* : pendant ces phases, toute nouvelle destination est refusée. Une annulation demandée pendant la phase **sending**, avant l'acceptation par Nav2, est mémorisée et transmise dès que le but est accepté. Si Nav2 refuse l'annulation, la phase revient à **navigating** avec un message explicite, afin de ne jamais laisser croire que le robot s'est arrêté.

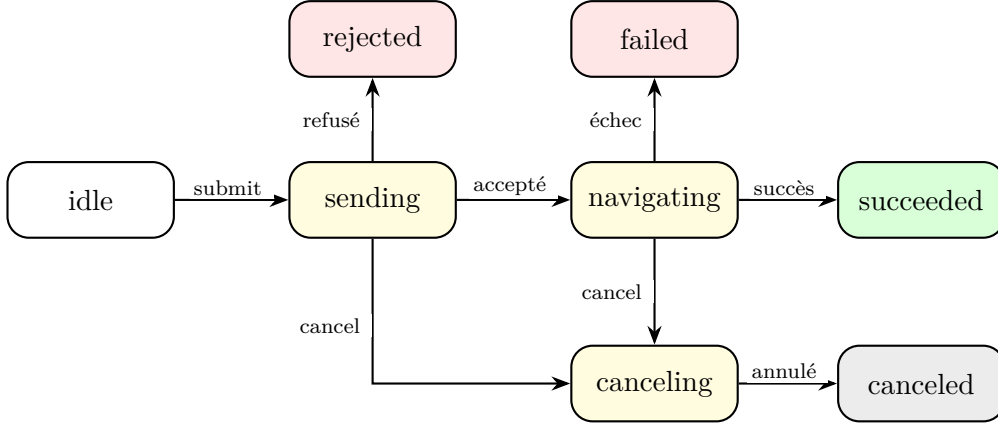


FIGURE II.3 – Phases d'une demande de navigation (en jaune, les phases actives).

5 Gestion des repères

Le dashboard dessine le monde à partir du fichier SDF de Gazebo ; ses coordonnées sont donc celles du monde simulé. Nav2, lui, travaille dans le repère **map** défini par la carte chargée. Ces deux repères ne coïncident pas nécessairement : une carte construite par SLAM a pour origine la position du robot au début de l'enregistrement.

La passerelle est configurée par la pose de l'origine du repère **map** dans le monde, notée (t_x, t_y, θ) (paramètres `world_offset_x`, `world_offset_y`, `world_offset_yaw`). La conversion d'une pose de la carte vers le monde est :

$$\begin{pmatrix} x_w \\ y_w \end{pmatrix} = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix} \begin{pmatrix} x_m \\ y_m \end{pmatrix} + \begin{pmatrix} t_x \\ t_y \end{pmatrix}, \quad \psi_w = \psi_m + \theta, \quad (\text{II.1})$$

où (x_m, y_m, ψ_m) est la pose dans **map** et (x_w, y_w, ψ_w) la pose dans le monde. Une destination choisie dans le dashboard est convertie par la transformation inverse avant d'être envoyée à Nav2 :

$$\begin{pmatrix} x_m \\ y_m \end{pmatrix} = \begin{pmatrix} \cos \theta & \sin \theta \\ -\sin \theta & \cos \theta \end{pmatrix} \begin{pmatrix} x_w - t_x \\ y_w - t_y \end{pmatrix}, \quad \psi_m = \psi_w - \theta. \quad (\text{II.2})$$

Le cap est manipulé en radians côté ROS et saisi en degrés dans l'interface. La position

du robot est toujours lue par TF dans le repère `map`, jamais à partir de l'odométrie seule, qui dérive.

6 Vérification des destinations

Avant d'envoyer un but à Nav2, la passerelle consulte la costmap globale. La destination, convertie dans `map`, est ramenée dans le repère de la grille (origine et rotation de la grille), puis la cellule est obtenue par

$$i = \left\lfloor \frac{g_x}{r} \right\rfloor, \quad j = \left\lfloor \frac{g_y}{r} \right\rfloor, \quad (\text{II.3})$$

où (g_x, g_y) sont les coordonnées dans la grille et r sa résolution. L'arrondi par défaut (et non la troncature) garantit qu'un point situé juste avant un bord négatif est bien considéré comme hors de la grille. La destination est refusée si la cellule est hors carte, inconnue (-1) ou si son coût atteint le seuil d'occupation de 65 (sur une échelle de 0 à 100). La carte approximative dessinée par le dashboard n'est jamais utilisée pour ce contrôle. Une cellule libre peut toutefois être inaccessible : dans ce cas, c'est Nav2 qui renvoie un échec, et l'interface l'affiche sans le transformer.

7 Règles de sûreté et de fraîcheur

Le tableau II.2 résume les règles appliquées de part et d'autre.

Côté	Règle
Passerelle	Refus si le temps simulé n'est pas utilisé (<code>use_sim_time</code> faux).
Passerelle	Refus si Nav2 est absent, si la transformation TF a plus de 2 s ou si la costmap a plus de 5 s.
Passerelle	Un seul but actif ; un <code>request_id</code> déjà reçu n'est jamais réexécuté.
Passerelle	Horloge de publication basée sur le temps simulé : l'état n'est plus publié si Gazebo est en pause.
Dashboard	État considéré périmé après 2,5 s : position retirée de la carte et commandes bloquées.
Dashboard	Commandes interdites en mode démonstration ou si le monde affiché diffère du monde simulé.
Dashboard	Délai d'acquiescement de 6 s ; en cas d'expiration, aucun renvoi automatique et message invitant à consulter l'état.

TABLEAU II.2 – Règles de sûreté et de fraîcheur des données.

Conclusion

La conception repose sur une passerelle ROS 2 unique, un protocole acquitté et un état publié en continu, ainsi que sur une conversion explicite entre le repère de la carte et celui du monde. Le chapitre suivant présente la réalisation de ces éléments.

Chapitre III

Réalisation

Introduction

Ce chapitre décrit la mise en œuvre de la conception : les paquets ROS 2 ajoutés ou modifiés, la passerelle de navigation, les composants du dashboard, l'outillage de lancement et les tests automatisés. Il se termine par les problèmes rencontrés lors de l'intégration sous Ubuntu et les corrections apportées.

1 Organisation des fichiers

Le travail s'insère dans le dépôt existant. Le tableau III.1 liste les principaux fichiers ajoutés.

Fichier	Rôle
navigation_msgs/srv/ SubmitNavigationGoal.srv	Interface du service de soumission.
navigation/scripts/ dashboard_navigation_node.py	Passerelle : services, action Nav2, TF, état JSON.
navigation/scripts/ dashboard_navigation_math.py	Conversions de repères et lecture de la costmap, sans dépendance ROS.
navigation/launch/ dashboard_navigation.launch.py	Lancement de la passerelle et, en option, de rosbridge.
navigation/test/ test_dashboard_navigation_math.py	Tests unitaires Python des conversions.
frontend/src/hooks/ useNavigation.js	Abonnement à l'état, fraîcheur, appels de services.
frontend/src/utils/ navigation.js	Validation du protocole et conditions d'envoi.
frontend/src/components/ NavigationPanel.jsx	Panneau de destination, d'état et d'annulation.
frontend/tests/ navigation.test.mjs	Tests unitaires JavaScript.
scripts/run-all.sh	Lancement de toute la chaîne en une commande.

TABLEAU III.1 – Principaux fichiers ajoutés.

Les fichiers existants modifiés sont principalement les `CMakeLists.txt` et `package.xml` des paquets `navigation` et `navigation_msgs` (génération de l'interface, installation des scripts, dépendances), la page `Map.jsx`, les composants de carte 2D et 3D, le hook de connexion ROS et la page des paramètres.

2 Passerelle ROS 2

2.1 Paramètres

La passerelle est un nœud Python (`rclpy`). Ses paramètres sont donnés dans le tableau III.2.

Paramètre	Défaut	Rôle
<code>world_id</code>	–	Monde du dashboard correspondant à la simulation
<code>base_frame</code>	<code>base_link</code>	Repère du robot lu dans TF
<code>world_offset_x, _y</code>	0,0	Position de l'origine de <code>map</code> dans le monde (m)
<code>world_offset_yaw</code>	0,0	Rotation de <code>map</code> dans le monde (rad)
<code>occupied_threshold</code>	65	Seuil de coût d'une cellule occupée
<code>use_sim_time</code>	<code>true</code>	Imposé par le fichier de lancement

TABLEAU III.2 – Paramètres de la passerelle.

2.2 Fonctionnement

Le nœud s'abonne à la costmap globale avec une qualité de service *transient local*, identique à celle de l'éditeur Nav2, afin de recevoir la dernière grille même s'il démarre après Nav2. Un temporisateur de 0,25 s calcule à chaque cycle la pose du robot et la disponibilité, puis publie l'état. Les principaux traitements sont :

- **soumission** : contrôle de l'identifiant, du but actif, du monde, des coordonnées, de la disponibilité, conversion par l'équation (II.2), contrôle de la cellule, puis envoi asynchrone du but à Nav2;
- **acceptation** : à la réponse de Nav2, passage en `navigating` ou en `rejected`, et envoi d'une annulation mémorisée le cas échéant;
- **retour** : mise à jour de la distance restante;
- **résultat** : conversion du statut Nav2 en `succeeded`, `canceled` ou `failed`;
- **annulation** : passage en `canceling` et demande d'annulation à Nav2; la phase terminale n'est affichée qu'à réception du résultat.

Les calculs de repères et de cellules sont isolés dans un module Python sans dépendance ROS, ce qui permet de les tester sans lancer de simulation.

2.3 Fichier de lancement

Le fichier `dashboard_navigation.launch.py` démarre la passerelle avec `use_sim_time` et inclut le serveur `roslaunch`, sauf si l'argument `start_roslaunch:=false` est donné. Les décalages de repère sont passés en arguments :

```
ros2 launch navigation dashboard_navigation.launch.py \
  world_id:=warehouse-harmonic \
  world_offset_x:=-1.2639 world_offset_y:=-3.9049 world_offset_yaw:=0.0
```

3 Interface web

3.1 Logique de navigation

Le hook React `useNavigation` s'abonne à l'état publié par la passerelle au moyen de `roslaunch`. Chaque message est validé par `parseNavigationState` : version, types des champs, phase connue et pose finie. Un message non conforme n'est pas affiché comme une donnée réelle. L'âge du dernier message est réévalué toutes les 0,5 s ; au-delà de 2,5 s, l'état est considéré périmé.

La fonction `navigationBlockReason` centralise les conditions d'envoi et retourne le motif de blocage à afficher (mode démonstration, déconnexion, données périmées, mauvais monde, passerelle non prête, but actif). Elle est réévaluée au moment exact du clic, et non seulement au dernier rendu. Un seul appel de service peut être en attente ; en l'absence de réponse sous 6 s, l'interface signale que le résultat est incertain et ne renvoie pas la commande.

3.2 Panneau de navigation et carte

Le composant `NavigationPanel` affiche la phase et son détail, la distance restante, le motif de blocage éventuel et les erreurs. L'opérateur choisit une destination en cliquant sur la carte 2D après avoir activé *Choose on map*, ou en saisissant X et Y en mètres et le cap en degrés. Les boutons *Go to destination* et *Cancel navigation* ne sont actifs que lorsque l'action est permise. Un message rappelle que fermer la page n'annule pas la navigation.

La page *Live map* a été adaptée (figure III.1) :

- le robot est dessiné à partir de la pose ROS uniquement lorsque l'état est frais et que le monde correspond ; la pose de la base de données n'est plus utilisée en remplacement ;
- un marqueur indique la destination choisie ;

- le mode démonstration est signalé explicitement et une erreur de l'API n'active plus implicitement les robots de démonstration ;
- la projection du robot et du trajet sur le plan 2D utilise la même transformation que le monde SDF, et l'orientation ROS est convertie pour le repère SVG ;
- la vue 3D n'applique plus de décalage arbitraire et oriente le robot selon sa pose ; le robot et le trajet décoratifs ont été supprimés.

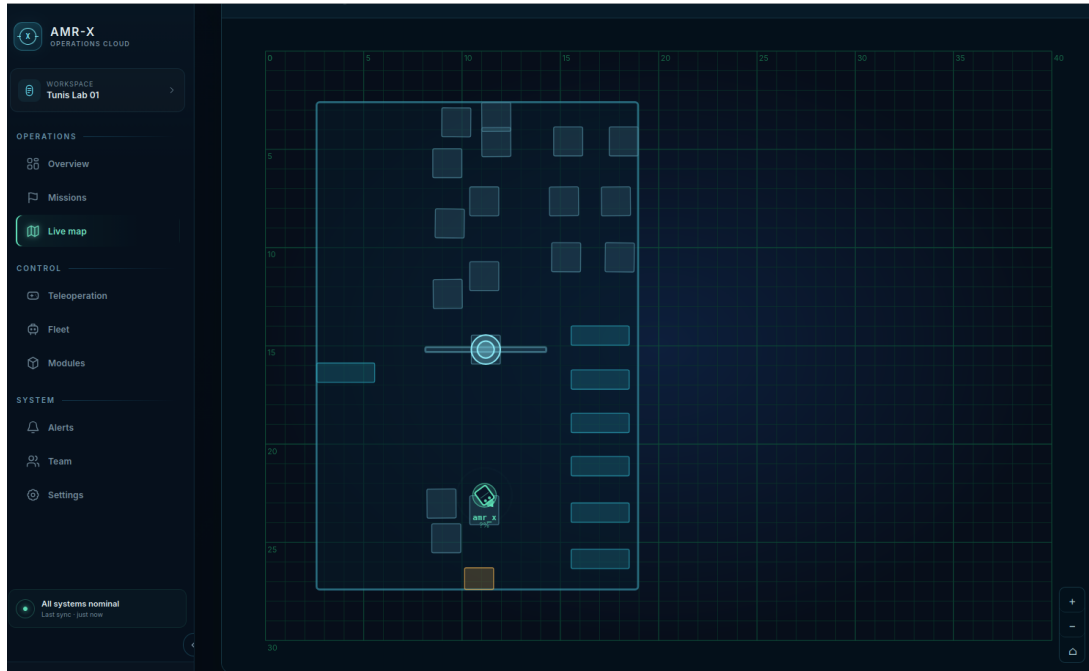


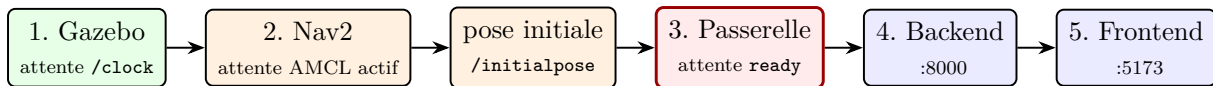
FIGURE III.1 – Page *Live map* : robot `amr_x` affiché à sa position ROS sur le plan du monde `warehouse_harmonic`.

Enfin, la connexion `rosbridge` se rétablit automatiquement après une fermeture du WebSocket, et une nouvelle adresse saisie dans *Settings* est appliquée sans recharger la page.

4 Lancement de la chaîne complète

La chaîne complète comporte cinq processus : simulation, Nav2, passerelle avec `rosbridge`, backend et frontend. Pour la rendre reproductible, un script `scripts/run-all.sh` les démarre dans l'ordre, attend que chacun soit prêt avant de lancer le suivant, et arrête l'ensemble par un simple `Ctrl+C` (figure III.2). Il envoie également la pose initiale à AMCL, ce qui évite une intervention manuelle dans RViz.

```
cd amr
bash scripts/run-all.sh # fenetre Gazebo ouverte
bash scripts/run-all.sh rviz # avec RViz en plus
```

FIGURE III.2 – Séquence de démarrage du script `run-all.sh`.

5 Tests automatisés

Deux jeux de tests unitaires ne nécessitent ni ROS ni Gazebo :

- **Python** (5 tests) : aller-retour entre les repères, repère carte tourné, cellules occupées, inconnues et hors carte, grille d'origine tournée, grille invalide ;
- **JavaScript** (5 tests) : conditions d'envoi, blocage des buts concurrents, rejet des états mal formés, validité des coordonnées nulles ou négatives, unicité et format des identifiants de requête.

Un test navigateur optionnel, fondé sur Playwright et un faux serveur rosbridge, vérifie en plus le parcours complet de l'interface (envoi, conversion degrés/radians, annulation, sélection sur la carte, blocage d'un mauvais monde). Il ne remplace pas les essais dans Gazebo présentés au chapitre suivant.

6 Problèmes rencontrés et corrections

Le code a d'abord été écrit sur un poste Windows, sans ROS. Son intégration sous Ubuntu avec la simulation réelle a révélé plusieurs problèmes, résumés dans le tableau III.3.

Conclusion

La passerelle, l'interface web et l'outillage de lancement ont été réalisés et intégrés à l'existant. Les tests unitaires couvrent les conversions de repères et la logique de l'interface. Il reste à vérifier le comportement de l'ensemble dans la simulation, ce qui fait l'objet du chapitre suivant.

Problème	Cause et correction
Arrêt de la passerelle au démarrage	L'attribut <code>self.handle</code> masquait la propriété <code>handle</code> de la classe <code>rclpy Node</code> , qui ne peut pas être modifiée. Renommage en <code>goal_handle</code> .
Échec du lancement sans <code>rosbridge</code> installé	Le chemin de <code>rosbridge</code> était résolu même avec <code>start_rosbridge:=false</code> . Utilisation de <code>FindPackageShare</code> , évalué seulement si l'inclusion est active.
Carte par défaut absente	<code>nav2.launch.py</code> référençait <code>amr_warehouse_map.yaml</code> , absent du dépôt. Carte par défaut remplacée par <code>warehouse_harmonic.yaml</code> .
Manifeste des mondes modifié	Chemins générés sous Windows avec des « \ ». Régénération sous Ubuntu.
Frontend non exécutable	Dépendances copiées depuis Windows, sans les binaires natifs Linux. Réinstallation par <code>npm ci</code> .
Absence de Docker	La base PostgreSQL du projet est lancée par Docker. Utilisation d'une base SQLite locale pour les essais.
Robot affiché à 4,1 m de sa position réelle	Décalage entre le repère de la carte et le monde Gazebo, détaillé au chapitre IV. Mesure et configuration des décalages de repère.

TABLEAU III.3 – Problèmes rencontrés lors de l'intégration et corrections.

Chapitre IV

Validation en simulation

Introduction

Ce chapitre présente la validation de la chaîne complète dans Gazebo. Il décrit l’environnement et la méthode d’essai, puis les vérifications préalables. Il expose ensuite le décalage de repères mis en évidence par les premiers essais et sa correction, avant de présenter les résultats mesurés après correction et les points qui restent à valider. Toutes les valeurs données sont issues de mesures réalisées le 29 septembre 2026.

1 Environnement et méthode d’essai

Les essais ont été réalisés sur un poste Ubuntu 24.04 avec ROS 2 Jazzy et Gazebo Harmonic, dans le monde `warehouse_harmonic`, avec la carte `warehouse_harmonic.yaml`. La chaîne complète était lancée par `scripts/run-all.sh`, fenêtre Gazebo ouverte (figure IV.1).

Pour obtenir des mesures reproductibles, les scénarios de navigation ont été pilotés par un script de test qui appelle *les mêmes services* que le dashboard, `/dashboard/navigation/submit` et `/dashboard/navigation/cancel`, et enregistre les changements de phase publiés par la passerelle. Les destinations libres et occupées sont tirées de la costmap globale. La vérité terrain est la pose du modèle `amr_x` lue directement dans Gazebo (`gz model -m amr_x -p`). L’erreur de position affichée est définie par

$$e = \sqrt{(x_{\text{passerelle}} - x_{\text{Gazebo}})^2 + (y_{\text{passerelle}} - y_{\text{Gazebo}})^2}, \quad (\text{IV.1})$$

où la pose de la passerelle est celle transmise au dashboard, dans le repère du monde.

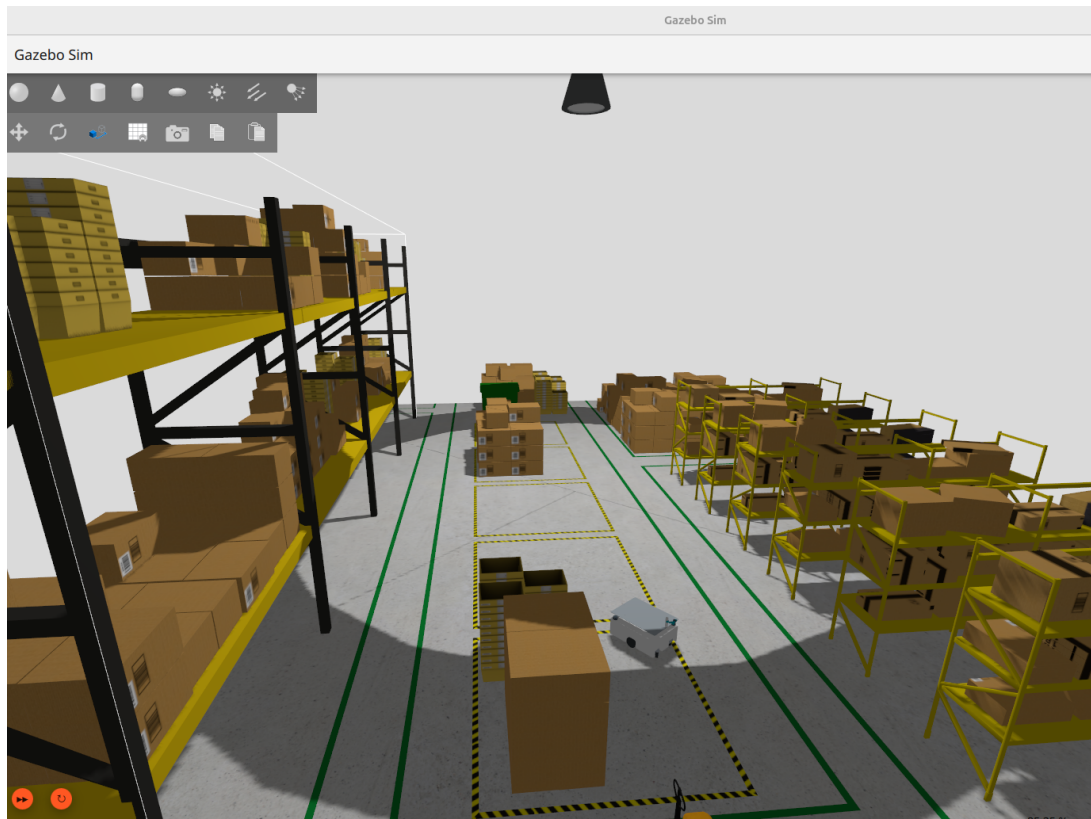


FIGURE IV.1 – Robot AMR-X simulé dans l'entrepôt Gazebo.

2 Vérifications préalables

Avant les essais en simulation, la compilation et les tests automatisés ont été exécutés sous Ubuntu (tableau IV.1).

Vérification	Résultat
Compilation <code>colcon</code> des paquets <code>navigation</code> et <code>navigation_msgs</code>	réussie
Génération de l'interface <code>SubmitNavigationGoal</code>	réussie
Tests unitaires Python des conversions	5/5 réussis
Analyse statique du frontend (<code>oxlint</code>)	aucune erreur
Tests unitaires JavaScript	5/5 réussis
Compilation de production du frontend (Vite)	réussie
Cohérence du manifeste des mondes SDF	à jour (6 mondes)

TABLEAU IV.1 – Vérifications préalables sous Ubuntu.

3 Mise en évidence d'un décalage de repères

3.1 Constat

Lors du premier essai depuis le dashboard, une destination a été choisie en (5,268; 2,448) sur la *Live map*. Nav2 a renvoyé un succès et le dashboard affichait le robot à destination, en (5,278; 2,423). Pourtant, dans Gazebo, le robot se trouvait en (3,953; -1,450), soit à environ 4,1 m de la destination demandée.

Pour caractériser ce défaut, la position transmise au dashboard a été comparée à la position Gazebo en trois endroits, les décalages de repère étant nuls (tableau IV.2). L'écart est pratiquement constant, d'environ 4,1 m, et l'erreur de cap reste inférieure à 0,6°. Il s'agit donc d'une translation pure entre les deux repères, et non d'une erreur de localisation aléatoire.

Position	Pose Gazebo (m)	Écart e (m)	Écart de cap (°)
Après le premier essai web	(3,953; -1,450)	4,094	-0,53
Après une navigation courte	(0,980; -2,931)	4,108	0,40
Après une annulation	(-4,877; 1,458)	4,131	-0,11

TABLEAU IV.2 – Écart entre la position affichée et la position Gazebo, sans correction de repère.

3.2 Analyse

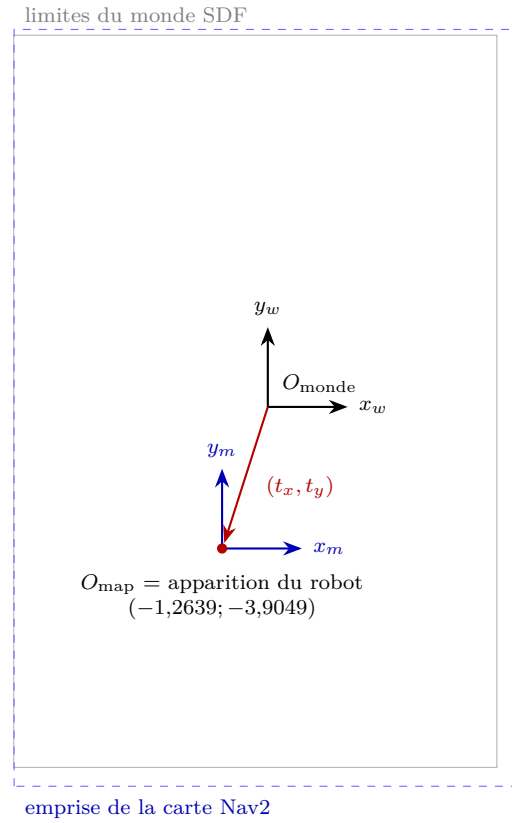
L'emprise de la carte `warehouse_harmonic.yaml` a été comparée aux limites du monde SDF. Le monde s'étend en x de -7,00 à 6,31 m et en y de -9,93 à 10,24 m, alors que la zone connue de la carte s'étend en x de -5,73 à 8,27 m et en y de -6,55 à 14,30 m. La carte est donc décalée d'environ (+1,3; +3,9) m par rapport au monde, ce qui correspond à l'opposé de la position d'apparition du robot, (-1,2639; -3,9049).

Cette observation s'explique par la façon dont la carte a été produite : une carte SLAM a pour origine la position du robot au début de l'enregistrement. L'origine du repère `map` se trouve donc à la position d'apparition du robot dans le monde Gazebo (figure IV.2). Le décalage étant nul, le dashboard dessinait des coordonnées `map` sur un plan exprimé dans le repère du monde, et les destinations cliquées étaient envoyées à Nav2 avec le même décalage.

3.3 Correction

Deux corrections ont été apportées au lancement :

- la passerelle reçoit la pose mesurée de l'origine de `map` dans le monde : $t_x = -1,2639$ m, $t_y = -3,9049$ m, $\theta = 0$;

FIGURE IV.2 – Origine du repère `map` dans le monde Gazebo.

- la pose initiale envoyée à AMCL est $(0; 0; 0)$ dans `map`, puisque le robot apparaît à l'origine de la carte.

4 Résultats après correction

4.1 Cohérence des positions

Après correction, l'écart entre la position transmise au dashboard et la position Gazebo a été mesuré en trois endroits (tableau IV.3). Il reste compris entre 0,6 et 5,6 cm, avec un écart de cap inférieur à 1° . La dernière mesure a été faite environ une seconde après une annulation en cours de déplacement.

Position	Pose Gazebo (m)	Écart e (m)	Écart de cap ($^\circ$)
Position d'apparition	$(-1,264; -3,905)$	0,014	0,33
Après la navigation B	$(-3,246; -1,698)$	0,006	0,40
Après l'annulation F	$(-0,016; -6,077)$	0,056	-0,99

TABLEAU IV.3 – Écart entre la position affichée et la position Gazebo, après correction.

4.2 Résultats par scénario

Le tableau IV.4 présente les résultats des scénarios exécutés. Les lettres reprennent la numérotation du protocole de validation du projet.

Réf.	Scénario	Résultat mesuré
A	Cohérence des positions	Écart de 0,6 à 5,6 cm et de moins de 1° en trois endroits (tableau IV.3).
B	Destination libre proche	But $(-3,30; -1,71)$, cap 90° : acquitté, phase navigating après 0,01 s, succeeded après 5,75 s. Erreur finale de position 5,6 cm ; cap final 101,0°.
D	Destinations invalides	Cellule occupée et point hors carte : refusés (« outside known free space or occupied »). Coordonnée non finie : refusée. Déplacement du robot pendant ces essais : 0,0 cm.
F	Annulation en route	Robot en mouvement (4,1 m parcourus 6 s après l'envoi) ; annulation à 7,4 s : canceling puis canceled en 0,03 s. Déplacement dans les 3 s suivantes : 3,6 cm.
G	Doublons et buts concurrents	Même request_id renvoyé : refusé (« already received »). Second but pendant la navigation : refusé (« a goal is already active »).
I	Simulation en pause	État publié à 3,6 Hz avant la pause, 0 message en 5 s pendant la pause, 3,8 Hz après la reprise, passerelle de nouveau prête.
J	Mauvais monde	Demande avec hospital-harmonic : refusée (« selected world does not match »).
–	Annulation sans but actif	Refusée (« no active destination »).

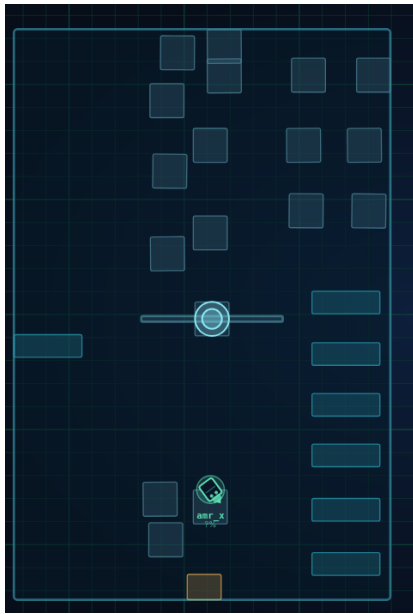
TABLEAU IV.4 – Résultats mesurés par scénario, après correction des repères.

4.3 Illustration d'une navigation réussie

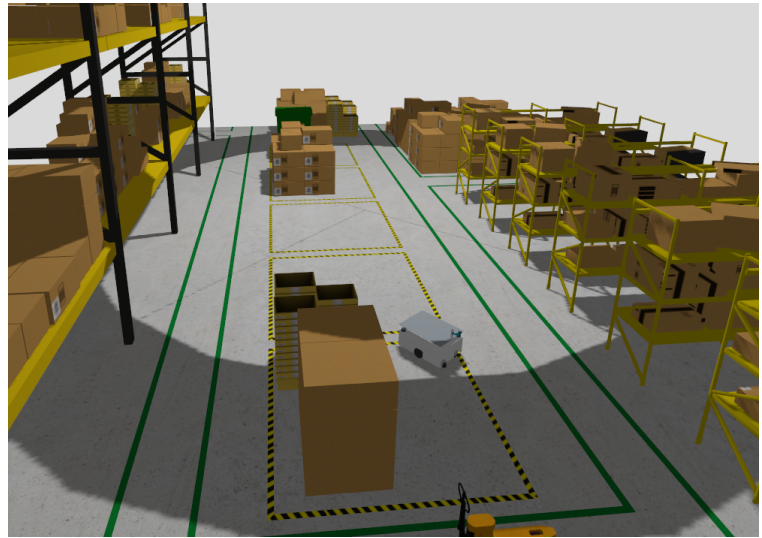
La figure IV.3 montre une navigation capturée simultanément dans le dashboard et dans Gazebo. Le robot, initialement en $(-0,05; -6,04)$, a reçu la destination $(-0,55; -3,71)$ par le service de soumission. Nav2 a renvoyé un succès après 10,8 s ; dans Gazebo, le robot s'est arrêté à 0,9 cm de la destination, et la position affichée dans le dashboard différait de 6,2 cm de la position Gazebo. Le dashboard montre le déplacement du robot et la trace de son trajet, et le panneau de flotte indique le résultat **succeeded** (figure IV.4).

4.4 Analyse

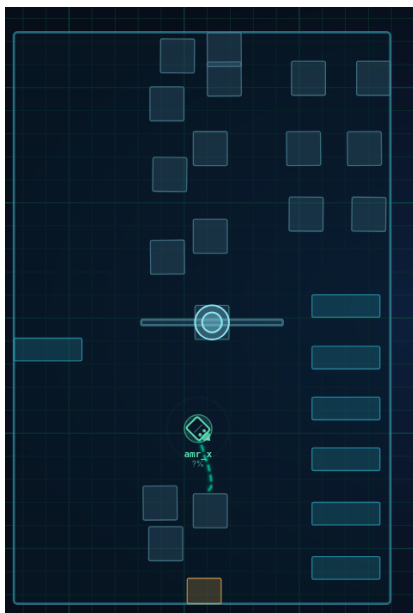
Les résultats montrent que la chaîne remplit les besoins fonctionnels : la destination est acquittée immédiatement, la phase suit fidèlement l'exécution de Nav2 et la position



(a) Dashboard, avant.



(b) Gazebo, avant.



(c) Dashboard, après.



(d) Gazebo, après.

FIGURE IV.3 – Navigation réussie : déplacement du robot vu dans le dashboard et dans Gazebo.

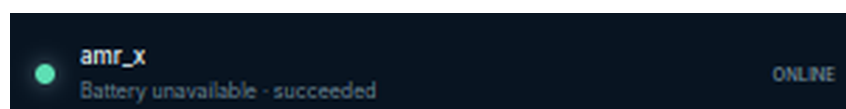


FIGURE IV.4 – Résultat de la navigation affiché dans le panneau de flotte du dashboard.

affichée correspond à celle du robot dans Gazebo à quelques centimètres près.

L'écart de cap final de 11° au scénario B reste inférieur à la tolérance angulaire configurée dans Nav2 (`yaw_goal_tolerance` de 0,25 rad, soit $14,3^\circ$); de même, l'erreur de position de 5,6 cm est inférieure à la tolérance de 0,25 m. Nav2 considère le but atteint dès que ces tolérances sont respectées; une précision plus fine relèverait du réglage de Nav2 et non du dashboard.

La fréquence de l'état observée en temps réel (3,6 à 3,8 Hz) est légèrement inférieure aux 4 Hz nominaux. Le temporisateur de la passerelle étant cadencé sur le temps simulé, cette fréquence suit la vitesse d'exécution de la simulation; c'est aussi ce qui provoque l'arrêt de la publication lorsque Gazebo est en pause, et donc le blocage des commandes dans le dashboard après 2,5 s.

La latence d'acquiescement mesurée (2 ms) a été obtenue par un client ROS local; elle n'inclut pas le passage par `rosbridge` et le navigateur.

5 Points restant à valider

Certains scénarios du protocole n'ont pas encore été exécutés dans la simulation :

- **C** – détour autour d'un obstacle : non mesuré spécifiquement ;
- **E** – destination libre mais inaccessible : à réaliser pour vérifier que l'échec de Nav2 est bien affiché ;
- **H** – coupure de `rosbridge`, puis reconnexion sans renvoi automatique ;
- **K** et **L** – mode démonstration et changement d'adresse dans *Settings*, vérifiés jusqu'ici par le test navigateur avec un faux serveur ROS uniquement ;
- cellule *inconnue* : la `costmap` globale utilisée ne contenait aucune cellule inconnue ; ce cas n'est couvert que par les tests unitaires.

Ces essais doivent être conduits depuis l'interface web, en notant pour chacun la configuration, le but, le résultat, la durée et les observations.

6 Limites

La solution est conçue pour la simulation. Elle ne gère qu'un robot et un monde à la fois, n'arbitre pas les commandes venant d'autres sources (RViz, téléopération) et n'ajoute pas d'authentification au niveau de `rosbridge` : elle n'est pas destinée à être exposée sur un réseau ouvert. Fermer la page ou perdre la connexion n'arrête pas la navigation en cours. Enfin, l'identifiant du monde et les décalages de repère sont configurés par l'opérateur au lancement; ils ne prouvent pas à eux seuls que le bon monde et la bonne carte sont chargés, d'où l'intérêt de la vérification de position décrite ci-dessus.

Conclusion

Les essais en simulation ont permis de valider l'envoi de destinations, le suivi de l'état, l'annulation, le refus des demandes invalides et la cohérence de la position affichée. Ils ont aussi mis en évidence un décalage de 4,1 m entre la carte Nav2 et le monde Gazebo, invisible sans comparaison avec la vérité terrain, qui a été analysé et corrigé.

Conclusion générale et perspectives

Ce stage avait pour objectif de permettre à un opérateur de commander le robot AMR-X depuis le dashboard web du projet et d'y suivre fidèlement sa position et le résultat de sa navigation. Au départ, le dashboard, la pile Nav2 et la simulation Gazebo fonctionnaient séparément.

Pour les relier, une passerelle ROS 2 dédiée a été conçue et réalisée. Elle reçoit les destinations par un service acquitté, vérifie chaque demande, dialogue avec Nav2 et publie un état de navigation unique et versionné. Côté dashboard, la page *Live map* permet désormais de choisir une destination sur la carte ou par saisie, de l'envoyer, de suivre la navigation et de l'annuler, tout en bloquant les commandes lorsque les données ne sont plus fiables. Un script de lancement rend la chaîne complète reproductible en une seule commande.

La validation en simulation a montré que la destination est acquittée immédiatement, que l'état affiché suit l'exécution réelle de Nav2, que les demandes invalides ou concurrentes sont refusées et que l'annulation arrête le robot. Elle a surtout mis en évidence un écart de 4,1 m entre la position affichée et la position réelle du robot, dû à la différence d'origine entre la carte SLAM et le monde Gazebo. Après mesure et configuration de ce décalage, la position affichée dans le dashboard correspond à celle du robot dans Gazebo à moins de 6 cm près. Ce résultat illustre l'importance de comparer l'interface à une vérité terrain plutôt que de se fier à la seule cohérence interne du système.

Plusieurs perspectives se dégagent de ce travail :

- terminer les scénarios restant à valider depuis l'interface (obstacle, destination inaccessible, coupure de rosbridge, mode démonstration, changement d'adresse) ;
- déterminer automatiquement le décalage entre la carte et le monde, par exemple en l'enregistrant avec la carte lors du SLAM, au lieu de le configurer à la main ;
- enregistrer les navigations lancées depuis la *Live map* dans la base de données, afin de les relier à la gestion des missions ;
- étendre la passerelle à plusieurs robots et à l'arbitrage entre sources de commande ;
- sécuriser l'accès à rosbridge avant tout usage en dehors d'un poste de simulation, puis préparer le passage sur le robot physique.